

Month 1 Foundations

Month 2 Core tools

Month 3 AI mathematics

From pre-university foundations to reading ML papers

90 days · 2 hours daily · 180 hours total · 12 weeks

Textbooks, videos, exercises, projects & AI connections

How to use this plan

This plan is designed for someone who already knows and has taught pre-university mathematics — functions, basic calculus, matrices, probability basics. The goal is not to start from zero but to upgrade that existing knowledge to the level needed to understand AI models mathematically, from the loss function all the way to the attention mechanism.

Daily rhythm	1 hour in the morning for new concepts. 1 hour in the evening for drills or coding. Keep them separate — fresh brain for new ideas, warmed brain for practice.
The "AI link" rule	Every time you learn a math concept, immediately ask: where does this appear in a neural network? Do not move on until you have answered this.
Notation notebook	Keep a dedicated notebook for notation. Every new symbol gets its own entry: what it means, an example, and the AI context where it appears.
Check with code	After every hand derivation, verify numerically in Python. If your gradient does not match PyTorch's autograd, your derivation is wrong. The code is the oracle.
Spaced repetition	On Sunday evenings, review the week's key equations from memory without looking at notes. This is the single most effective study technique.
Do not skip drills	The drill sessions feel boring but they build fluency. You cannot think clearly about a concept if you are still slow at the mechanical operations.

Tag legend

Concept	Drill	AI link	Project	Review
New theory or concept	Pen-and-paper exercises	Direct AI application	Build something in code	Consolidate and test

Foundations

You already know pre-university math — this month upgrades it to university level and re-frames every concept through the lens of AI. No ideas from scratch, just extensions of what you already taught. By the end of Month 1 you will be comfortable with the notation and rigour of university mathematics.

Week 1	Numbers, sets & logic The language all math is written in	
Mon–Tue	Sets, functions & notation Domain, codomain, range. Injective / surjective / bijective functions. This is how we describe what a model does: it maps an input space to an output space.	Concept
Wed	Real numbers & sequences Supremum, infimum, convergence of sequences. Needed to understand when gradient descent converges and what "the loss decreases" really means.	Concept
Thu	50 pen-and-paper exercises Function composition, sequence limits, set operations. Use any standard pre-calculus problem set.	Drill
Fri	Functions as models Write $f(x) = wx + b$ in set notation. What is the domain? The codomain? What does "learning w " mean in function terms?	AI link
Sat–Sun	Review + preview Summarise your notes. Watch 3Blue1Brown "Essence of Calculus" episodes 1–3 as a visual preview for next week.	Review
AI payoff: Every neural net is a function $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$. You need to read and write that notation fluently.		

Week 2	Single-variable calculus Derivatives, integrals, limits	
Mon–Tue	Limits & continuity — rigorously epsilon-delta definition. Why it matters: loss landscapes have flat regions and near-discontinuities that trip up training.	Concept
Wed–Thu	Differentiation: all rules Power, chain, product, quotient rules. Geometric meaning: slope of tangent line. This is gradient descent in one dimension.	Concept
Fri	Differentiate 30 functions Include compositions like $f(g(x))$ — this is exactly the chain rule that powers backprop.	Drill
Sat	Gradient descent in 1D by hand Minimise $f(w) = w^2 - 4w + 5$ using $w \leftarrow w - \alpha \cdot f'(w)$. Run 10 steps with $\alpha=0.1$. Watch it converge.	AI link
Sun	Integrals: overview only Integrals are less central to deep learning but understand them as area under a curve. You will need them for probability in Month 2.	Review
AI payoff: The derivative of the loss w.r.t. one weight is the simplest possible form of backpropagation.		

Week 3	Multivariable calculus Partial derivatives, gradient, chain rule	
Mon–Tue	Partial derivatives $\text{del}(f)/\text{del}(x)$ holds y constant. $f(x,y) = x^2 * y$ gives $\text{del}(f)/\text{del}(x) = 2xy$. This answers: "how does the loss change if we tweak weight w_3 by a tiny amount?"	Concept
Wed	The gradient vector $\text{grad}(f) = (\text{del}(f)/\text{del}(x_1), \dots, \text{del}(f)/\text{del}(x_n))$. Points in the direction of steepest ascent. Gradient descent moves in the opposite direction.	Concept
Thu	Multivariable chain rule $\text{del}(L)/\text{del}(w) = (\text{del}(L)/\text{del}(a)) * (\text{del}(a)/\text{del}(w))$. One equation. This IS backpropagation — every layer applies this rule.	Concept
Fri	Compute gradients by hand For $f(x,y,z) = x^2 * y * z + \sin(y)$, find $\text{grad}(f)$. Then compute the gradient of a 1-layer network loss by hand.	Drill
Sat–Sun	Backprop from scratch in NumPy Implement forward + backward pass for a 2-layer net. Verify your hand-computed gradients match PyTorch autograd output exactly.	Project
AI payoff: A neural net has millions of weights. The gradient is the partial derivative of the loss w.r.t. every weight simultaneously.		

Week 4	Linear algebra I Vectors, matrices, linear transformations	
Mon	Vectors: operations & geometry Addition, scaling, dot product, angle between vectors. The dot product is the similarity score used in attention mechanisms.	Concept
Tue	Matrices: operations Addition, multiplication, transpose, inverse. Matrix multiply = a linear transformation of space. Each layer transforms data into a new representation.	Concept
Wed	Linear transformations geometrically What does multiplying by a matrix actually do to space? Stretch, rotate, reflect, project onto a subspace.	Concept
Thu	Matrix multiplication drills 3x3 examples by hand. Verify $AB \neq BA$. Prove $(AB)^T = B^T * A^T$. Compute inverses for 2x2 matrices.	Drill
Fri	One forward pass by hand Take W (3x2), x (2x1), b (3x1). Compute $y = Wx + b$ step by step. Apply ReLU. This is one neural network layer, completely by hand.	AI link
Sat–Sun	3Blue1Brown linear algebra series Watch "Essence of Linear Algebra" episodes 1–7. The best visual intuition available. Take notes on transformations.	Review
AI payoff: Each neural network layer computes $y = Wx + b$. Matrix multiplication IS the computation.		

Core tools

Month 1 gave you the language. Month 2 gives you the heavy machinery — the math that directly powers every modern AI model. Every single topic this month maps 1-to-1 to something in PyTorch or a real ML paper. By the end you will understand why loss functions look the way they do.

Week 5	Linear algebra II Eigenvalues, SVD, PCA	
Mon–Tue	Eigenvalues & eigenvectors $A\mathbf{v} = \lambda\mathbf{v}$. Directions a matrix only scales, never rotates. Critical for understanding covariance matrices and training stability.	Concept
Wed	Singular value decomposition (SVD) $A = U * \Sigma * V^T$. Every matrix decomposes into rotate → scale → rotate. The sigma values measure how much information each dimension carries.	Concept
Thu	PCA from first principles Center data → covariance matrix → eigenvectors → project. This is linear dimensionality reduction. Your autoencoder does the nonlinear version.	Concept
Fri	PCA on toy data by hand 2D dataset of 5 points. Compute mean, covariance matrix, eigenvectors. Project onto the first principal component manually.	Drill
Sat–Sun	PCA vs autoencoder on MNIST Apply sklearn PCA (linear) and your PyTorch autoencoder (nonlinear) to MNIST. Visualise compressed representations. Compare reconstruction quality.	Project
AI payoff: Autoencoders learn a nonlinear version of PCA. SVD underlies how transformers compress information into latent representations.		

Week 6	Probability I Foundations, distributions, Bayes	
Mon	Sample spaces, events, axioms Kolmogorov axioms. Joint, marginal, conditional probability. $P(A B) = P(A \text{ and } B) / P(B)$.	Concept
Tue	Random variables & expectation Discrete and continuous RVs. $E[X]$, $\text{Var}[X]$. The loss function is $E[\text{loss over the data distribution}]$ — training approximates this expectation.	Concept
Wed	Key distributions Bernoulli, Gaussian, Categorical, Uniform. Know their PDFs, means, variances. Softmax outputs a categorical distribution. MSE assumes Gaussian noise.	Concept
Thu	Bayes theorem deeply $P(w \text{data})$ is proportional to $P(\text{data} w) * P(w)$. Posterior, likelihood, prior. This is the foundation of all probabilistic machine learning.	Concept
Fri	30 probability exercises Conditional probability, Bayes problems. Make them concrete: "given a positive test, what is the true probability of disease?"	Drill

Sat–Sun	Distributions in ML Softmax -> categorical distribution. MSE loss -> Gaussian noise assumption. Binary cross-entropy -> Bernoulli. Derive each from maximum likelihood estimation.	AI link
AI payoff: Every model output is a probability distribution. Loss functions are derived from probability theory, not invented arbitrarily.		

Week 7	Information theory Entropy, cross-entropy, KL divergence, MLE	
Mon–Tue	Entropy $H(p) = -\sum(p \cdot \log p)$ How much uncertainty is in a distribution? Uniform distribution = maximum entropy. One-hot = zero entropy. This quantifies information.	Concept
Wed	Cross-entropy & KL divergence $H(p,q) = -\sum(p \cdot \log q)$. $KL(p q) = \sum(p \cdot \log(p/q))$. Cross-entropy loss = KL divergence + constant. Now you know why it works.	Concept
Thu	Maximum likelihood estimation (MLE) Find parameters θ that maximise $P(\text{data} \theta)$. Log-likelihood turns products into sums. Both MSE and cross-entropy derive from MLE.	Concept
Fri	Derive loss functions from MLE Assume Gaussian noise -> derive MSE. Assume categorical output -> derive cross-entropy. Do both completely by hand with pen and paper.	Drill
Sat–Sun	Visualise entropy and cross-entropy Plot $H(p)$ for Bernoulli as p varies 0 to 1. Show cross-entropy between two distributions. Connect to your CIFAR-10 training loss curves.	Project
AI payoff: Cross-entropy loss IS the negative log-likelihood of a categorical distribution. KL divergence IS the VAE regularisation term.		

Week 8	Optimisation Convexity, gradient descent variants, Adam	
Mon	Convex functions & loss landscapes f is convex if $f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda)f(y)$. Convex = one global minimum. Neural nets are non-convex but still trainable.	Concept
Tue	Gradient descent: theory $w \leftarrow w - \alpha \cdot \text{grad}(L)$. Learning rate α controls step size. Too large: diverge. Too small: slow. Understand the geometry.	Concept
Wed	SGD, Momentum, RMSProp, Adam SGD = one sample at a time. Momentum = add velocity. Adam = adaptive per-parameter learning rates using 1st and 2nd moment estimates of gradients.	Concept
Thu	Optimise functions by hand Minimise $f(x,y) = x^2 + 2y^2$ using gradient descent. Run 5 steps with $\alpha=0.1$. Plot the convergence path. Observe the elliptical trajectory.	Drill
Fri	Implement Adam from scratch $m = \beta_1 m + (1-\beta_1) \cdot \text{grad}(L)$. $v = \beta_2 v + (1-\beta_2) \cdot \text{grad}(L)^2$. $w = w - \alpha \cdot m_{\text{hat}} / (\sqrt{v_{\text{hat}}} + \epsilon)$. Test on a quadratic loss.	AI link
Sat–Sun	Loss landscape intuition Study saddle points, flat regions, local minima. Why does SGD noise help escape saddle points? Why do wide minima generalise better?	Review

AI payoff: Training a neural net is an optimisation problem. The choice of optimiser determines whether and how fast training converges.

AI mathematics

Months 1 and 2 built pure mathematics. Month 3 applies every tool directly to AI architectures. Each week targets one model family. By the end you can open any ML paper, read the equations, and understand what they mean. The capstone week ends with reading the original Transformer paper.

Week 9	Neural networks: complete math	
	Forward pass, backprop, activations	
Mon	Activation functions mathematically $\sigma(x) = 1/(1+e^{-x})$, $\tanh$, $\text{ReLU} = \max(0, x)$. Derivatives: $\sigma' = \sigma(1-\sigma)$. Why ReLU avoids vanishing gradients where sigmoid does not.	Concept
Tue	Forward pass as function composition $f(x) = a_n(W_n * a_{n-1}(W_{n-1} * \dots * a_1(W_1 * x + b_1) \dots + b_{n-1}) + b_n)$. Write it fully for a 3-layer network and trace one input through.	Concept
Wed	Backprop: full derivation Derive $\partial(L)/\partial(W_l)$ using chain rule layer by layer. Introduce δ_l = the error signal. Derive the 4 fundamental equations of backpropagation.	Concept
Thu	Backprop by hand on 2-layer net Sigmoid activations, MSE loss, 2 inputs, 1 output. Compute forward + backward pass completely by hand, then verify against PyTorch autograd.	Drill
Fri	Vanishing & exploding gradients If $\sigma' < 0.25$ per layer, after 10 layers the gradient shrinks by 0.25^{10} . This is why deep nets were nearly untrainable before ReLU and batch normalisation.	AI link
Sat-Sun	Full net in pure NumPy Build a 3-layer net with forward + backward in NumPy only. Train on XOR. No PyTorch. Document every array shape and every equation you use.	Project
AI payoff: This week you fully derive every equation in a neural network using the math you now have. No black boxes remain.		

Week 10	Convolutional networks	
	Convolution operation, pooling, backprop through Conv	
Mon	Convolution as a mathematical operation $(f * g)(t) = \int f(\tau)g(t-\tau) d\tau$. Discrete 2D: $(I * K)[i, j] = \sum_m \sum_n I[i+m, j+n] * K[m, n]$. K is the kernel/filter.	Concept
Tue	Stride, padding, output size $\text{Output size} = \text{floor}((n + 2p - f) / s) + 1$. Receptive field = what input region influences one output neuron. Stacking layers grows the receptive field.	Concept
Wed	Backprop through convolution Gradient w.r.t. input = full convolution with flipped kernel. Gradient w.r.t. kernel = valid convolution with input. Derive both from chain rule.	Concept
Thu	Convolution by hand on 5x5 image Apply a 3x3 Sobel edge-detection kernel manually. Compute the output feature map cell by cell. Observe which edges are detected.	Drill

Fri	Why CNNs work: the math perspective Weight sharing = same kernel applied everywhere = translation equivariance by construction. Hierarchical features = composed convolutions with growing receptive fields.	AI link
Sat–Sun	Annotate your CIFAR-10 autoencoder Re-read every line of your Week 1 autoencoder code. Annotate each operation with its mathematical form. What are the shapes? What does each layer compute?	Project
AI payoff: CNNs = signal processing mathematics applied to images. Convolution is a rigorous mathematical operation you can fully derive.		

Week 11	Attention & transformers Dot-product attention, softmax Jacobian, positional encoding	
Mon	Scaled dot-product attention $\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$. Q, K, V are linear projections of the input. Dividing by $\sqrt{d_k}$ prevents softmax saturation in high dimensions.	Concept
Tue	Softmax: the math $\text{softmax}(x)_i = e^{x_i} / \sum_j e^{x_j}$. Converts raw scores to probabilities. Fully differentiable. The Jacobian of softmax is a full matrix: $d \cdot \text{softmax}_i / d \cdot x_j$.	Concept
Wed	Multi-head attention Run h attention functions in parallel on projected subspaces. $\text{MultiHead} = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^O$. Each head can attend to different relationships.	Concept
Thu	Positional encoding $\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{2i/d})$. Adds position information since attention has no inherent order. Sinusoids allow the model to generalise to longer sequences.	Concept
Fri	Attention by hand on toy example Take Q, K, V each 3x2. Compute $Q \cdot K^T$, scale by $1/\sqrt{2}$, apply softmax row by row, multiply by V. Full attention output completely by hand.	Drill
Sat–Sun	Single-head attention in NumPy Build $\text{Attention}(Q, K, V)$ from scratch in NumPy. Test: does it attend to similar tokens? Visualise the attention weights as a heatmap using matplotlib.	Project
AI payoff: Transformers run the entire world of modern AI. The math is just matrix multiplications and softmax — you now have all the tools to read it.		

Week 12	Capstone week Connect everything, read a real paper, plan ahead	
Mon	Draw your full concept map Sets -> Functions -> Calculus -> Gradients -> Linear Algebra -> Probability -> Optimisation -> Neural Nets -> CNNs -> Attention. Connect every arrow.	Review
Tue	Regularisation mathematically L2: adds $\lambda \ w\ ^2$ to loss = Gaussian prior on weights. Dropout: expected network = ensemble of 2^n subnetworks. Batch norm: normalises layer inputs.	Concept
Wed	Bias-variance tradeoff & generalisation Overfitting = memorised training data. Why wide minima generalise better than sharp minima. The double-descent phenomenon in modern overparameterised models.	Concept
Thu–Fri	Read "Attention Is All You Need" (2017) Vaswani et al., arXiv:1706.03762. Read sections 3 and 4 carefully. You now understand every equation. Write a 1-page summary in your own words.	Project

Sat	Mini end-to-end project Pick any model (VAE, transformer, CNN). Derive the full math, implement from scratch in NumPy, verify against PyTorch. Document every equation used.	Project
Sun	Reflect and plan the next 3 months What can you now read that you could not 3 months ago? Natural next steps: measure theory, Riemannian geometry, Bayesian deep learning, information geometry.	Review
AI payoff: Real ML papers are combinations of everything you learned. This week you read one and understand every equation in it.		

Textbooks & resources

Every book listed below is either free online or available on Library Genesis (libgen.rs). You do not need to buy anything. The priority label tells you when to read each one.

Months 1–2 (primary)

Mathematics for Machine Learning

Marc Peter Deisenroth, A. Aldo Faisal, Cheng Soon Ong

Cambridge University Press, 2020

Linear algebra, analytic geometry, matrix decompositions, vector calculus, probability, optimisation. Written specifically to bridge math and ML.

[FREE PDF: mml-book.github.io](https://mml-book.github.io)

When: PRIMARY — use this as your main reference throughout months 1 and 2.

Linear Algebra Done Right

Sheldon Axler

Springer, 4th edition (free online)

The most rigorous and elegant linear algebra textbook. Go here when MML is not deep enough.

[FREE PDF: linear.axler.net](https://linear.axler.net)

When: Supplementary for weeks 4 and 5.

Calculus (Early Transcendentals)

James Stewart

Cengage, any edition

Comprehensive single and multivariable calculus with hundreds of exercises. Use chapters 1–4 (single variable) and chapters 11–14 (multivariable).

[Available on Library Genesis \(libgen.rs\) — search title](#)

When: Supplementary for weeks 2 and 3.

Introduction to Probability

Dimitri Bertsekas & John Tsitsiklis

Athena Scientific, 2nd edition

Rigorous but approachable probability theory. Excellent problem sets. Covers exactly what is needed for ML probability.

[Available on Library Genesis — search "Bertsekas probability"](#)

When: Supplementary for weeks 6 and 7.

Month 3 (AI-specific)

Deep Learning

Ian Goodfellow, Yoshua Bengio, Aaron Courville

MIT Press, 2016

Chapters 2–4 cover all the math. Chapters 6–9 cover neural nets, regularisation, CNNs. The standard reference textbook for deep learning.

[FREE online: deeplearningbook.org](https://deeplearningbook.org)

When: PRIMARY for month 3. Read chapters 2–4 first, then follow the weeks.

Neural Networks and Deep Learning

Michael Nielsen

Online book, 2019

The best walkthrough of backpropagation mathematics anywhere. Chapter 2 derives all 4 equations of backprop with full intuition.

[FREE: neuralnetworksanddeeplearning.com](https://neuralnetworksanddeeplearning.com)

When: Essential for week 9. Read chapter 2 completely.

Attention Is All You Need

Vaswani, Shazeer, Parmar et al.

NeurIPS 2017 (original transformer paper)

The paper that started modern AI as we know it. Sections 3–4 contain the full attention math. 15 pages total.

FREE: arxiv.org/abs/1706.03762

When: Read during week 12 (capstone). You will understand every equation.

The Matrix Calculus You Need for Deep Learning

Terence Parr & Jeremy Howard

Online paper, 2018

A focused 33-page paper on exactly the matrix calculus used in backprop. Fills the gap between standard calculus textbooks and ML practice.

FREE: arxiv.org/abs/1802.01528

When: Read at the start of month 3 as a bridge from months 1–2.

Video resources (free)

3Blue1Brown — Essence of Calculus

Grant Sanderson

YouTube (3blue1brown channel), 2017

12 videos. The best visual intuition for derivatives, chain rule, and gradients. Watch before the corresponding week.

youtube.com/@3blue1brown — "Essence of Calculus" playlist

When: Watch episodes 1–3 at end of week 1. Episodes 4–8 during week 2.

3Blue1Brown — Essence of Linear Algebra

Grant Sanderson

YouTube (3blue1brown channel), 2016

16 videos. Transformations, eigenvectors, dot products all shown geometrically. Transforms how you see matrices forever.

youtube.com/@3blue1brown — "Essence of Linear Algebra" playlist

When: Watch during weeks 4 and 5.

3Blue1Brown — Neural Networks

Grant Sanderson

YouTube (3blue1brown channel), 2017

4 videos. Visual backpropagation derivation. Watch this before week 9 as motivation, then derive everything yourself.

youtube.com/@3blue1brown — "Neural Networks" playlist

When: Watch before week 9.

Andrej Karpathy — Neural Networks: Zero to Hero

Andrej Karpathy (ex-OpenAI, Tesla)

YouTube (karpathy channel), 2023

Builds backprop, neural nets, and a GPT from scratch in Python. Code-first companion to the theory you are learning.

youtube.com/@AndrejKarpathy — "Neural Networks: Zero to Hero"

When: Watch in parallel with month 3, especially weeks 9–11.